

Software Quality Engineering (SE3002)

Assignment 02 — 100 marks

Structural Testing and Coverage Analysis of PX4 Autopilot v1.17.0 — SE3002 Assignment 02

PX4-Autopilot v1.17.0 · d6f12ad1c4f70ad3230afd7d86e971421e02fef4

Ali Haider Bajwa (24i3102) · Taimoor Khalid (24i3052) · Ashar Ahmed
(24i3072)

Section C (SE)

1. Introduction and testing approach

We performed white-box structural testing of three selected PX4 v1.17.0 production areas, chosen from direct source reading rather than from any coverage target: `src/lib/hysteresis/hysteresis.cpp` (a boolean state machine with asymmetric time delays, tested at GTest unit level), `src/lib/pid/PID.cpp` (a control-loop PID with saturation, anti-windup, and degenerate-input guards, GTest unit), and `src/lib/battery/battery.cpp` (state-of-charge fusion, internal-resistance estimation, warning and fault classification, and remaining-time estimation, GTest functional). The Battery class doubles as our MC/DC component, because its warning and fault outputs drive failsafe behavior. Our obligations were 100% statement coverage and 100% decision and branch coverage over this scope wherever practically achievable, with every residual gap individually investigated and justified, plus MC/DC independence pairs for the applicable compound decisions of the critical component, recorded with each atomic condition and each overall decision showing both outcomes in the workbook matrix. Upstream tests were read and run to learn project conventions and to establish a baseline, but every claimed test is student-authored, as the reconstructing patch proves. No production behavior was changed to gain coverage; the entire diff consists of test registrations and three new test files.

2. Local setup and fixed baseline

All work was cloned, built, tested, and measured locally on a student-owned machine. The repository was cloned recursively at the fixed tag v1.17.0 (`git clone --branch v1.17.0 --recursive https://github.com/PX4/PX4-Autopilot.git`), and the local HEAD was verified as `d6f12ad1c4f70ad3230afd7d86e971421e02fef4`, so every coverage figure in this report traces to that baseline plus our own test-only changes. The environment was Linux Mint 22.3 (Ubuntu 24.04 base) on x86_64, with gcc/g++ 13.3.0, cmake 3.28.3, ninja 1.11.1, lcov 2.0-1 (gcov 13.3.0), Python 3.12.3, and git 2.43.0. One environment note: the top-level `make tests` wrapper was bypassed because its `cmake-build` macro mishandles paths (an unquoted `$(SRC_DIR)`); we configured and built directly with `cmake` and `ninja` (§10), which is equivalent and is what all evidence was produced with — this is a build-invocation detail, not a production change. The baseline runs green: upstream `unit-Hysteresis` passes 7 of 7 and upstream `unit-PID` passes 4 of 4, with logs in `evidence/baseline/` and baseline scope coverage in `evidence/baseline/scope_baseline_branch.info`.

3. Repository analysis and scope selection

We began from the production code. The hysteresis class holds a boolean state with independent per-direction delay windows and is driven entirely through caller-supplied timestamps, which makes it fully deterministic and ideal for unit testing; its two-sided time-guarded state machine (`set_state_and_update` plus `update`) contains about seven decisions, including two compound direction guards, so it is genuinely non-trivial despite its small size. The PID class sits on a control-critical path with output and integral saturation, conditional integration, a non-finite-integral guard, and a time/history guard on its derivative term — again deterministic through its direct API, and squarely unit-testable. The Battery class is the richest target: it fuses voltage-based and coulomb-counted state of charge, runs a recursive-least-squares internal-resistance estimator, classifies warnings across a four-rung ladder, detects over-voltage faults, and estimates remaining flight time. It depends on PX4 parameters (`BAT_*`), uORB topics (`vehicle_status`, `flight_phase_estimation`), and high-resolution time, which is exactly what the GTest functional level exists for; notably, no upstream test links this library, so its baseline coverage was zero and authorship is clean. We deliberately excluded `SlewRate` (branch-free math with nothing to decide), `circuit_breaker` (a single superficial `&&` wrapper the brief rejects as a target), and all GUI, generated, and test-only code, while refusing to exclude difficult business logic to flatter the percentage.

4. Structural test derivation

Statement obligations were discharged by 61 student tests: 11

`HysteresisStructural` unit tests covering decisions H-D1 through H-D8, including exact `>=`-expiry boundaries the upstream suite never probes; 33

`BatteryStructural` functional tests covering B-D1 through B-D12 across the warning ladder, spike-fault detection, the RLS-reset gate, state-of-charge fusion, the discharge time-delta clamp, scale fallback, publish paths, and the armed and fixed-wing remaining-time gates; and 17 `PIDStructural` tests covering P-D1 through P-D3 plus saturation edges, from the update-integral flag through NaN and negative-infinity guards, the derivative-guard quad, the exact `FLT_EPSILON` boundary, negative-dt robustness, windup clamping, output-limit edges, and a zero-gain degenerate configuration. Every test maps to its obligations in workbook Sheet 1 (61 rows: HST-01–11, BST-01–33, PST-01–17), and both outcomes of every reachable decision are demonstrated, with MC/DC independence pairs in §5 and workbook Sheet 2 (32 rows). Edge, boundary, invalid, error, and state-transition

cases follow the source wherever it implies them: timestamp exactness and repeat-request stamp keeping in hysteresis; NaN, infinity, degenerate-dt, and saturation limits in PID; threshold exactness, the 1.05 over-voltage boundary, stale-phase handling, unconnected and zero-cell states in Battery. Execution is fully green: upstream 7/7 and 4/4 at baseline, and student suites 11/11, 17/17, and 33/33, with logs in evidence/logs/. No failure was manufactured; eight development-time failures were investigated and all traced to wrong test-side expectations (L130 gate timing twice, a NaN output value, two D-term signs, and three saturation-freeze assumptions), corrected without touching production code. The PID file deserves an authorship note: it was authored by Ashar Ahmed from his own derivation (branch `ashar-pid-work` with a living handoff document) and verified on Linux by the lead, who corrected five expectations against the v1.17.0 source — the D-term sign is PLUS (PID.cpp line 47), and no conditional-integration freeze gate exists (lines 57–64) — reaching a final 17/17 pass, with the original delivery preserved for provenance.

5. MC/DC analysis of the critical component

Battery was selected as the MC/DC component because its warning and fault classification directly drives failsafe behavior: a missed EMERGENCY warning risks loss of the vehicle, while a spurious one aborts the mission. The critical behavior under analysis is the warning-ladder and spike-fault classification together with the guards that arm them — the `updateBatteryStatus` chain, `determineWarning`, `determineFaults`, and the remaining-time reset and update gates. Five non-trivial compound decisions implement or directly govern this behavior and were analyzed: the four-condition RLS-reset gate at line 130, the two-condition over-voltage fault at lines 327–328, the three-condition filter-reset disjunction at lines 370–371, the two-condition armed-update guard at line 375, and the three-way fixed-wing level-flight gate at lines 377–378. Trivial logging, wrapper, and routing decisions elsewhere were excluded with reasons recorded at derivation time. Workbook Sheet 2 carries the full per-decision evidence — atomic-condition values, overall outcome, independence pair, demonstrated condition, and source/test reference — thirty-two rows in total. The hysteresis direction guards are shown through armed-versus-quiescent and fired-versus-held pairs at exact expiry boundaries; the four-condition reset gate is shown by flipping each condition while the other three hold (unconnected, pre- versus post-initialization, cleared resistance flag, zero cells), observed as an open-circuit-voltage estimate of 16.0 versus 0.0 with zero current; the spike fault is shown including the exact 1.05

boundary that proves the comparison is strict; the filter reset is shown through near-zero-state and consumed-transition pairs, where a moved reset target (5.0 to 7.0) discriminates a genuine hold from a re-reset; the armed guard is shown through unarmed-versus-armed and finite-versus-NaN-current pairs; the fixed-wing gate is shown through multirotor-versus-fixed-wing, fresh-versus-stale stamp, and level-versus-climb pairs; and the PID derivative guard is shown through its time-by-history quad with the exact epsilon boundary. All short-circuit, state, parameter, and message setup needed to reach each condition is explicit in the test bodies — synthetic timestamps, direct parameter control, and fixture-held uORB publications so subscriptions actually receive data. The honest reading of these pairs is that every atomic condition can independently flip its decision's outcome on the real code path that arms warnings, faults, and remaining-time updates, so no condition in this failsafe-critical classification logic is dead or masked — except for two cases investigated in §7 that are unreachable by construction rather than by weak testing.

6. Test implementation and execution

Hysteresis and PID tests use `px4_add_unit_gtest`, linking only their libraries, while the Battery suite uses `px4_add_functional_gtest`, which additionally links the functional main, the PX4 layers, uORB, and parameters. Upstream test registrations were left untouched, and there was no upstream battery test to disturb. Every test follows setup–run–check discipline with a freshly constructed object per case, so there is no inter-test state and no ordering dependence: hysteresis and PID drive the public API directly with fixed synthetic timestamps (including exact plus-or-minus-one-microsecond boundaries), while Battery uses a test subclass exposing protected parameters, deterministic thresholds, and fixture-held uORB publications — a design forced by a real discovery that helper-local publications unadvertise on destruction and silently break subscriptions. CMake and registration changes in the patch are three added lines across the hysteresis, PID, and battery build files (plus two lines for the functional registration). Authorship is proven by `patch/full_reconstruction.patch`, a clean `git diff` against v1.17.0 containing 1,140 insertions across six files with zero production-line edits. Execution evidence is complete: baseline logs show upstream 7/7 and 4/4, student logs show 11/11, 17/17, and 33/33, and all five binaries build and run in both the normal and the instrumented coverage configurations from the documented commands.

7. Coverage measurement and gap analysis

Coverage was captured before our tests (baseline) and after (final) with lcov and gcov branch coverage, and both machine-readable reports and full HTML are submitted. Hysteresis stands at 22 of 22 lines in both runs with 17 of 20 branch arcs; the three missing arcs are provably unreachable because the operand combinations contradict the line-75 entry guard, as proven by inspection. PID stands at 22 of 22 lines with all 10 of 10 arcs, closing the two non-finite-integral and derivative-guard sub-cases the upstream baseline left open at 8 of 10. Battery, absent from the baseline entirely since no upstream test links the library, reaches 230 of 231 lines (99.6%) with all 19 functions covered and 174 of 248 branch arcs. Our suite's contribution is therefore the entire battery coverage from a zero baseline, plus the hysteresis boundary and MC/DC obligations and the PID sub-arcs upstream never exercised. The remaining gaps are few and each is named precisely. The sole uncovered line is battery line 72, an error log for a missing parameter handle that cannot fire with intact v1.17.0 metadata and would require a corrupted parameter store. The hysteresis arcs above are unreachable by construction. The B-D9 non-finite-average arm is unreachable through the public API because its only writer is itself guarded by the B-D10 finiteness check — defense in depth subsumed downstream. The L356 copy-failure path is not deterministically reachable because it requires a topic withdrawal between the update check and the copy inside a single call; our unadvertised-topic test documents safe behavior there instead. Branch totals also include roughly seventy compiler-generated exception and unwind arcs from matrix, filter, and parameter templates that correspond to no source decision and cannot be driven from the public API; they are reported separately and never excluded. Nothing here is hand-waved, and difficult logic was never omitted to improve a percentage. On MC/DC completeness: every applicable decision — H-D5, H-D7, the four-condition B-D4, B-D8, B-D9's second and third conditions, all of B-D10 including the armed-plus-NaN pair, all of B-D11, and the P-D3 quad — is demonstrated with independence pairs, with only the unreachable B-D9 first condition justified rather than shown.

8. Findings, limitations, and improvements

Eight student-test failures occurred during development, and each was investigated before any defect claim — setup, expected values, and dependency behavior first — and each resolved as a wrong test-side expectation with production code untouched: two inverted models of the L130 reset gate's pre-init firing; one NaN output where IEEE semantics give NaN for infinite feedback while

the integral guard itself held; two D-term sign errors against the PLUS in PID line 47; and three saturation expectations written against a freeze gate that does not exist in v1.17.0. No reproducible production defect was found, and none is manufactured to fill a quota — the suite is green because the covered logic met its specification on the exercised paths. Our conclusions stay limited to what was analyzed and executed. Residual risks are: only three of thousands of PX4 files are covered, so failsafe consumers, estimators, controllers, and drivers are outside the evidence; the L72 corrupted-parameter path is untested by construction; estimator convergence was shown only on synthetic steps, never on real noisy flight data; fixed-wing remaining-time paths used synthetic phase stamps rather than estimator output; and all timing is synthetic, so real-time scheduling effects are out of scope. Concretely, we recommend replaying long-horizon current and voltage traces through state-of-charge estimation, adding a scratch-store test for the corrupted-parameter path, covering the fixed-wing gate with estimator-produced phase data in simulation, and upstreaming this first-ever Battery functional test so its zero baseline never returns. An independent consistency audit by Taimoor Khalid (audit report, gap analysis with per-arc BRDA classification, packaging checklist in taimoor/) verified hashes, counts, patch applicability, and every gap claim against the evidence; its findings were each re-verified and either closed with new probe tests or absorbed into this report.

9. Final quality judgment

The structural evidence supports a narrow but strong claim: the three analyzed components behave as specified on every executable path reachable locally. Hysteresis reaches 22/22 lines with all reachable branches taken, including exact timer-expiry boundaries the upstream suite never probes. PID reaches 22/22 lines and 10/10 branch arcs, closing the non-finite-integral and derivative-history gaps upstream leaves open. Battery — which had no upstream test and therefore 0% baseline coverage — reaches 230/231 lines with all 19 functions covered, and independence pairs prove every atomic condition of the failsafe-critical warning, fault, and remaining-time gates can flip its decision, except cases proven unreachable by construction rather than by weak tests. Eight student-test failures during development were each traced to wrong expectations, never to production defects, and no reproducible production defect is claimed; the suite is green because the covered logic met its specification on the exercised paths, and no failure was manufactured. Confidence is therefore high but strictly scoped: statement coverage is complete over the analyzed scope and every reachable

decision shows both outcomes, so residual risk concentrates in the individually justified gaps — the L72 corrupted-param log, three hysteresis arcs contradicting the entry guard, and the subsumed B-D9 non-finite arm. Everything outside the scope remains unsupported: the failsafe consumers of these signals, the estimators feeding them, real-sensor noise, real-time scheduling, and the rest of the PX4 codebase, none of which this evidence addresses. In particular, RLS convergence was shown only on synthetic steps and fixed-wing remaining-time behavior only on synthetic phase stamps. The honest reading is that the selected business logic is structurally sound and deterministically verified at unit and functional levels, while any claim about PX4 as a whole would require system, simulation, and flight evidence far beyond this assignment.

Methodologically, the suite follows a strict setup–run–check discipline with fresh objects per case, fixed synthetic timestamps, and hand-computed expectations, so every passing test is a reproducible white-box obligation discharge rather than an observational anecdote.

10. Reproduction instructions

```
# 1. Baseline (inside PX4-Autopilot @ d6f12ad, see §2 for clone/setup)
git clone --branch v1.17.0 --recursive https://github.com/PX4/PX4-Autopilot.git
cd PX4-Autopilot && git rev-parse HEAD # expect
d6f12ad1c4f70ad3230afd7d86e971421e02fef4
bash Tools/setup/ubuntu.sh # PX4 toolchain (sudo; use a space-free path)

# 2. Normal test build + run (all commands from the repo root)
cmake . -G Ninja -DCONFIG=px4_sitl_test -B build/px4_sitl_test
cd build/px4_sitl_test
ninja unit-Hysteresis unit-HysteresisStructural unit-PID unit-PIDStructural
functional-BatteryStructural
./unit-Hysteresis && ./unit-HysteresisStructural && ./unit-PID && ./unit-
PIDStructural && ./functional-BatteryStructural

# 3. Coverage build + measurement (separate build dir, instrumented)
cd ../../ && cmake . -G Ninja -DCONFIG=px4_sitl_test -DCMAKE_BUILD_TYPE=Coverage -B
build/px4_sitl_coverage
cd build/px4_sitl_coverage && ninja unit-Hysteresis unit-HysteresisStructural unit-
PID unit-PIDStructural functional-BatteryStructural
./unit-Hysteresis; ./unit-HysteresisStructural; ./unit-PID; ./unit-
PIDStructural; ./functional-BatteryStructural
lcov --capture --branch-coverage --directory . --gcov-tool gcov --ignore-errors
mismatch -o /tmp/cov_all.info
lcov --branch-coverage --extract /tmp/cov_all.info "*/src/lib/hysteresis/*"
"*/src/lib/pid/*" "*/src/lib/battery/*" --ignore-errors mismatch -o
scope_coverage_branch.info
genhtml --branch-coverage scope_coverage_branch.info -o html
```

11. AI-assistance record

AI tooling assisted repository navigation, build troubleshooting, test scaffolding, and coverage analysis throughout, but every AI-suggested test, expectation value, and coverage claim was independently verified by compiling, running, and measuring — eight wrong AI-assisted expectations were caught and corrected during development (two Battery reset-gate timings, one PID NaN output, two Ashar PID sign expectations refuted by PID line 47, and three Ashar saturation expectations refuted by the absence of a freeze gate in lines 57–64). Scope, MC/DC pairs, and gap justifications were derived from direct source reading rather than model assertions. Ashar's record: OpenCode and Muse Spark for navigation, float32 verification scripting, scaffolding, and handoff drafting, with introduced assumptions (brief line and decision framing, fallback sign expectations) corrected through direct source reading, hand recomputation, and an independent float32 model. Taimoor's record: OpenCode for the cross-file consistency audit, coverage-gap classification from raw branch data, report prose drafting, viva-bank derivation, and the packaging checklist, writing no PX4 or test code; his line-to-arc mapping risk was contained by citing raw branch-data strings, and every one of his findings was re-verified before merging. No raw chat transcripts are submitted, and no production code was AI-modified — the diff contains only test registrations and new test files. The full per-iteration log is `AI_ASSISTANCE_LOG.md`.

12. References and evidence index

- Brief: `Assignment02-SQE.docx.md` · Plan: `WORKFLOW_PLAN.md` · Progress: `PROGRESS_LOG.md`
- Workbook: `workbook/` (proposed submission copy: `Workbook_proposed.xlsx`, 2 sheets) · Patch: `patch/` · Coverage: `evidence/baseline/`, `evidence/final/` · Logs: `evidence/logs/` · Teammate deliveries: `ashar/`, `taimoor/`
- Official: PX4 Repository, Developer Environment, Unit Tests, Testing & CI, Simulation, v1.17 Release Notes (per brief).